Health Insurance Claims Simulator

Project guide: purpose, flow, screens and database schema

A classroom web app where a patient or hospital submits a health-insurance claim, claims staff review it, and an administrator records a simulated payment. Every person, plan, rule and amount is fictional.

Flask **SQLAlchemy ORM** **MySQL** **Jinja templates** **Bootstrap**

Based on the MITRAz Skills BRD v1.0 (16 Oct 2026) | Simulator only: not real insurance advice

1. Purpose of the project

Why it exists, what it does, and what it deliberately does not do

The problem

A patient who is treated in hospital needs a way to submit a claim and see what happens next.

A hospital desk may ask for cashless treatment; a policyholder may ask for reimbursement afterwards.

Claims staff must check the policy, ask for missing papers, decide, and explain the decision.

Without a system these hand-offs are untraceable.

What the app does

One traceable application for the whole journey:

- Login with 4 roles (holder, hospital desk, officer, admin)
- Create, edit and submit claims
- Document checklist (received / missing)
- Officer review: assign, ask for info, approve or reject with a reason
- Payout calculation with deductible and co-pay
- Simulated settlement and ledger
- Full history of every status change
- Search, filters, dashboards, reports, CSV export

What it is NOT

A simulator for learning. It does not:

- issue real policies or give insurance advice
- diagnose or judge medical necessity
- connect to hospitals or insurers
- move real money, send SMS or e-mail
- store real medical files or personal data

All plans, limits, formulas and people are invented classroom rules, not IRDAI rules.

What a second student must be able to do with the finished app

Log in as each role -> move one claim through a valid workflow -> see a reasoned result with the calculation -> prove that an unauthorised account cannot change another person's claim.

2. The four roles

Each account has one role. The server checks the role on every request; hiding a button is never the only protection.

Policyholder

aasha, bilal, chitra ...

CAN DO

+ See only own policies, members and claims
+ Create a reimbursement draft and submit it
+ Mark documents received / missing
+ Answer an information request

MUST NOT

x See or edit another holder's claim
x Approve or mark paid

Hospital desk

city_hospital, lotus_hospital ...

CAN DO

+ See cashless claims of its own hospital
+ Create and submit cashless claims for that hospital
+ Provide checklist information

MUST NOT

x Submit for another hospital
x Approve / reject
x See unrelated claims

Claims officer

officer_meera, officer_rahul

CAN DO

+ See all submitted claims in a work queue
+ Assign a claim to self, start review
+ Request more information
+ Approve (eligible amount + reason) or reject (reason)

MUST NOT

x Edit master policies
x Approve a draft
x Take a claim another officer holds

Administrator

admin_demo

CAN DO

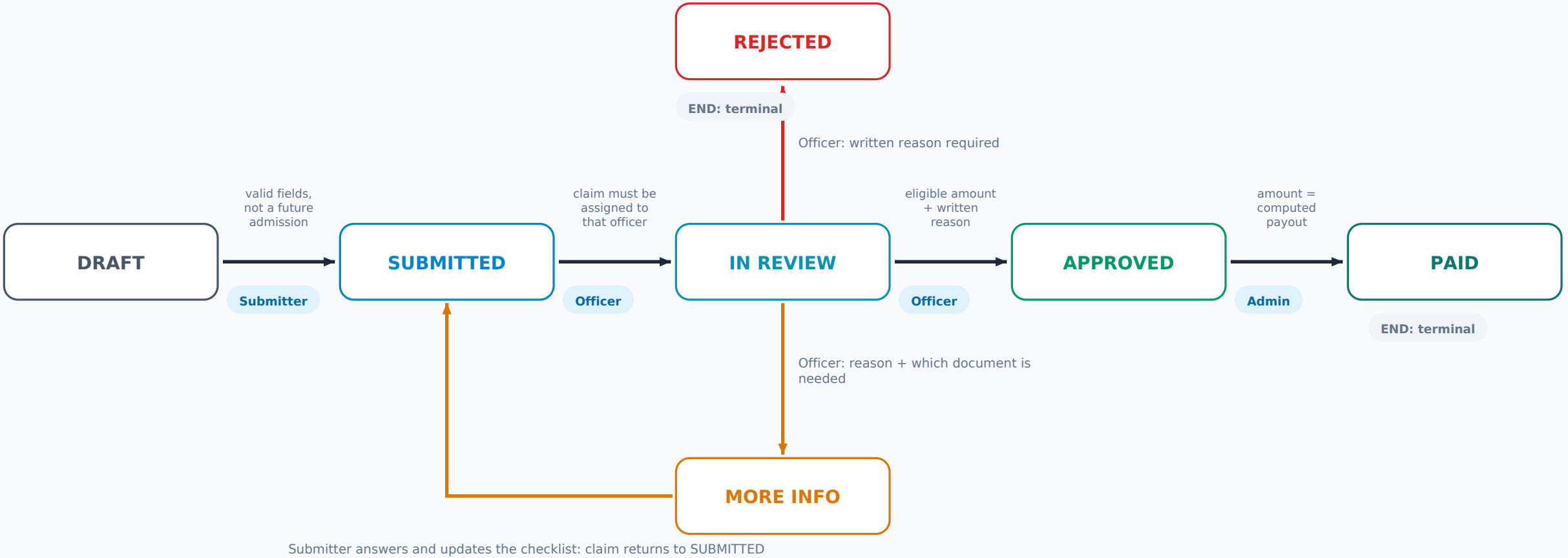
+ Manage hospitals, plans, user roles
+ Record the simulated settlement of an approved claim
+ View all reports

MUST NOT

x Change a claim outcome without a history event
x Bypass server checks

3. Claim lifecycle: the states and who moves a claim

Each arrow is one allowed transition. Anything else is refused by the server.



Every arrow writes history

Actor, previous status, new status, time and a note are saved in claim_history in the same database transaction as the status change.

Invalid jumps are refused

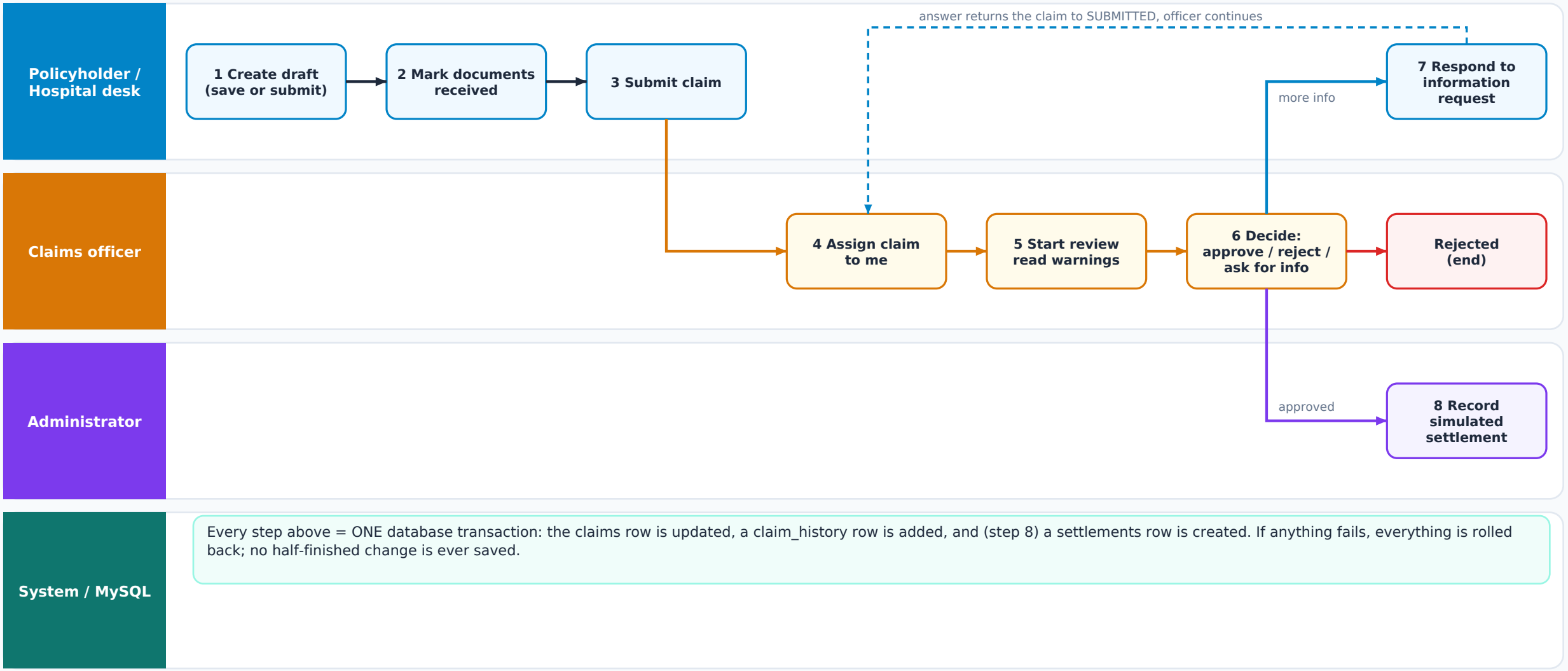
Example: submitted -> approved, or editing a rejected claim. A forged request gets an error; the status stays unchanged.

Who owns each step

Submitter = the user who created the claim. Officer = the officer assigned to it. Admin = any administrator. Drafts are private to their creator.

4. End-to-end journey (who does what)

Follow the numbers 1 to 8; arrows show the hand-off between people



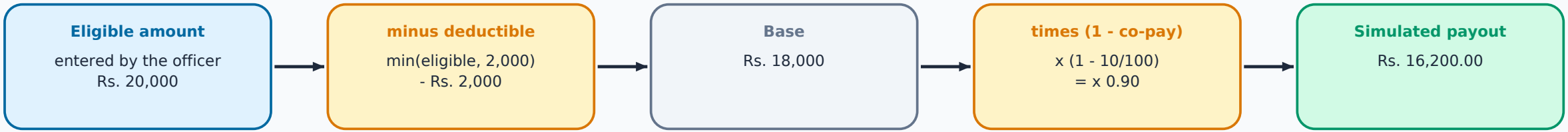
Cashless: submitted by the hospital desk, payee of the settlement is the hospital. **Reimbursement: submitted by the policyholder, payee is the policyholder.**

Nothing is approved automatically: an officer always reads the warnings and writes a reason.

5. Business rules: payout and officer checks

All numbers are invented classroom rules; money uses exact decimals, never floats

Payout formula (example: eligible 20,000 | deductible 2,000 | co-pay 10%)



The officer can only enter an eligible amount that is above 0, not above the requested amount, and not above the remaining cover (annual limit minus earlier paid payouts). The screen shows a live preview; the server recalculates.

What the officer's review page warns about (the officer decides, nothing is automatic)

1 Policy active?
Policy status active AND admission date between start and end date (inclusive).

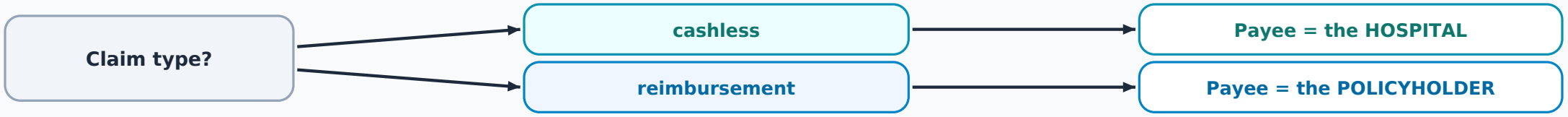
2 Treatment covered?
Plan covers inpatient / daycare? e.g. Essential Care covers inpatient only.

3 Possible duplicate?
Same member, hospital, dates and claim type as another claim: warning only.

4 Documents?
Hospital bill and discharge summary received or missing.

5 Cover left?
Annual limit minus simulated payouts already recorded for the policy.

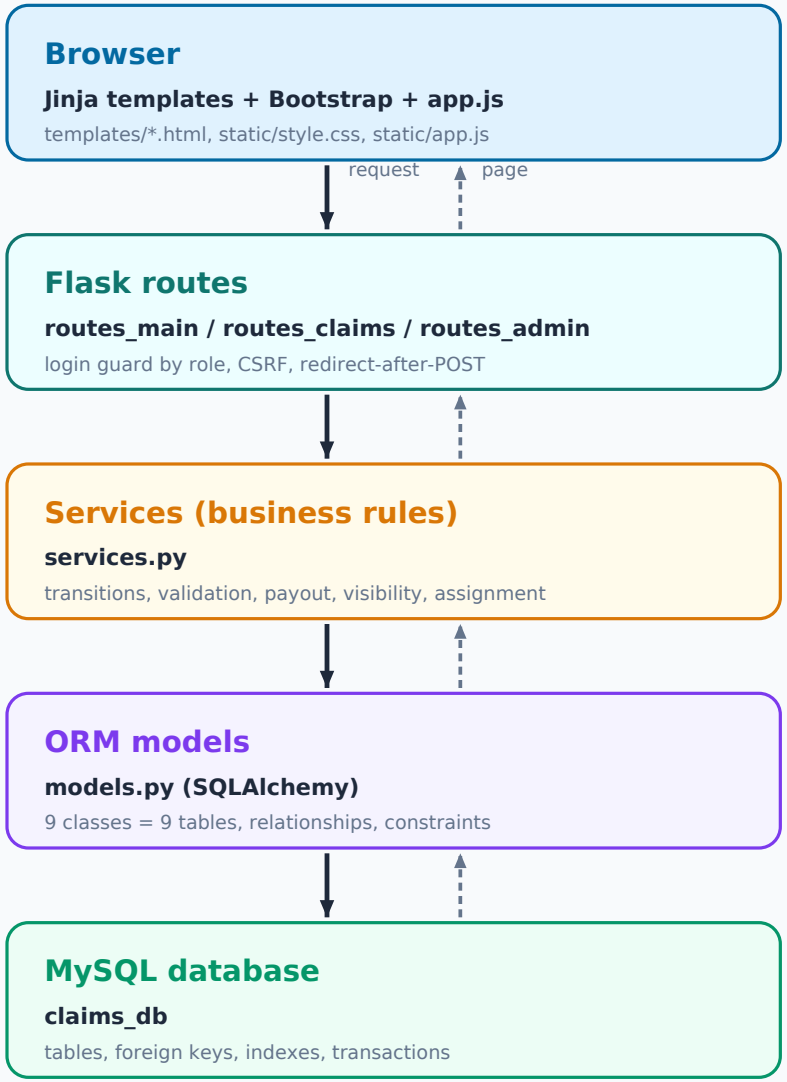
Who gets paid



Cashless is allowed only when the hospital is in the network AND the logged-in desk belongs to that hospital. Reimbursement may use any listed hospital. Settlement happens only for an APPROVED claim, once, and the amount must equal the computed payout. No real money moves.

6. How the application is built

A request travels down the layers; the answer travels back up as an HTML page



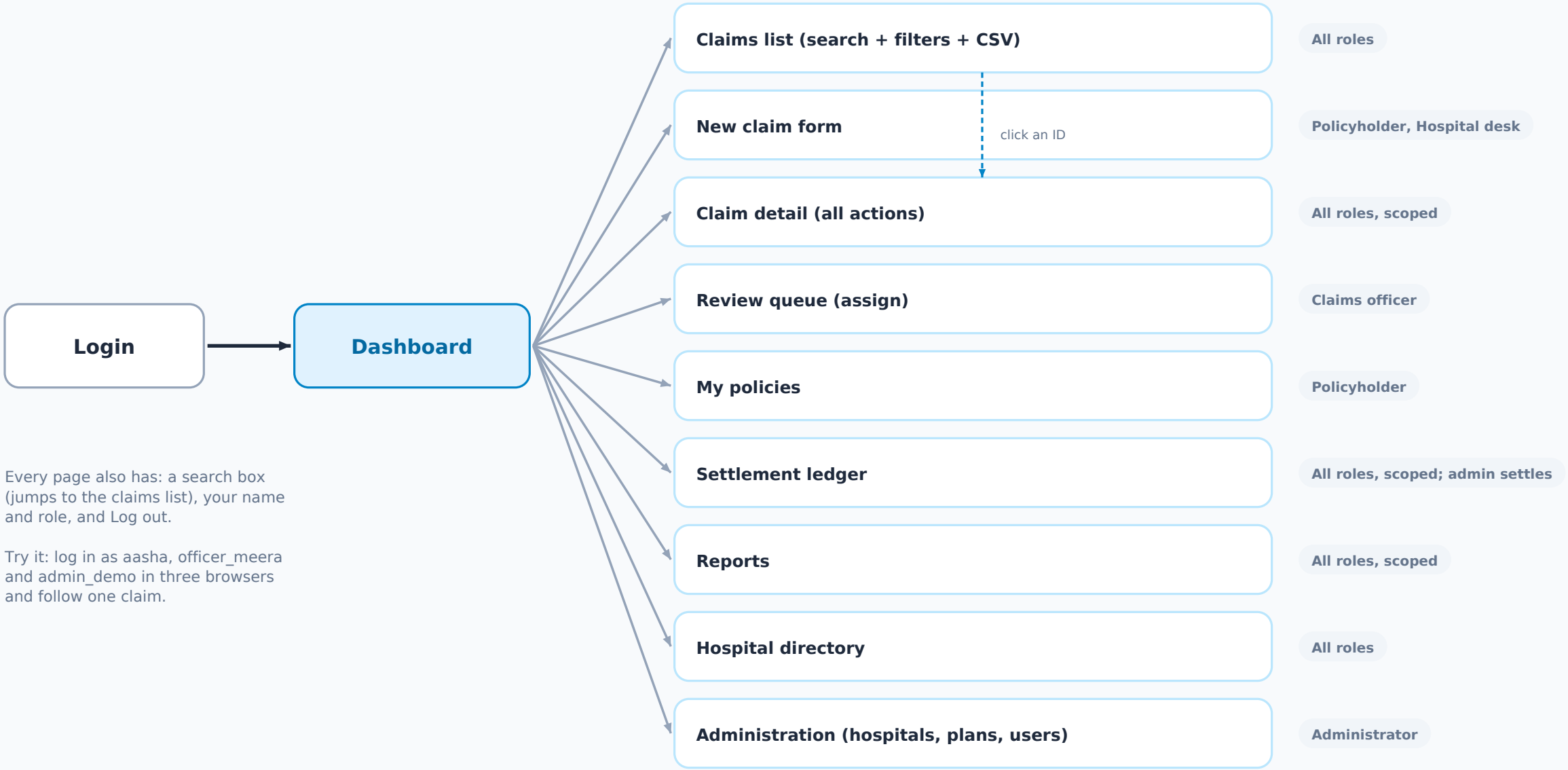
Project folder

`run.py / wsgi.py`	start the app (python run.py)
`config.py`	database login + settings from environment
`app/__init__.py`	creates the Flask app, registers parts
`app/auth.py`	who is logged in; role guard decorator
`app/routes_main.py`	login, dashboard, policies, hospitals
`app/routes_claims.py`	claim list, form, detail, all workflow buttons, CSV
`app/routes_admin.py`	admin pages, reports, settlement ledger
`app/services.py`	ALL business rules in one readable place
`app/models.py`	the database tables as Python classes
`app/seed.py`	imports the sample CSV files into MySQL
`schema.sql / data/`	MySQL DDL and the 9 synthetic CSV files
`tests/test_app.py`	16 automated tests of the BRD scenarios

Why routes are thin: a route only checks the role, calls ONE service function inside a transaction (commit on success, rollback on error) and redirects. The rules live in services.py, so the same checks apply to every request, even a forged one.

7. Screen map: where each role can go

Arrows = links you click. The menu shown on the left of every page depends on your role.



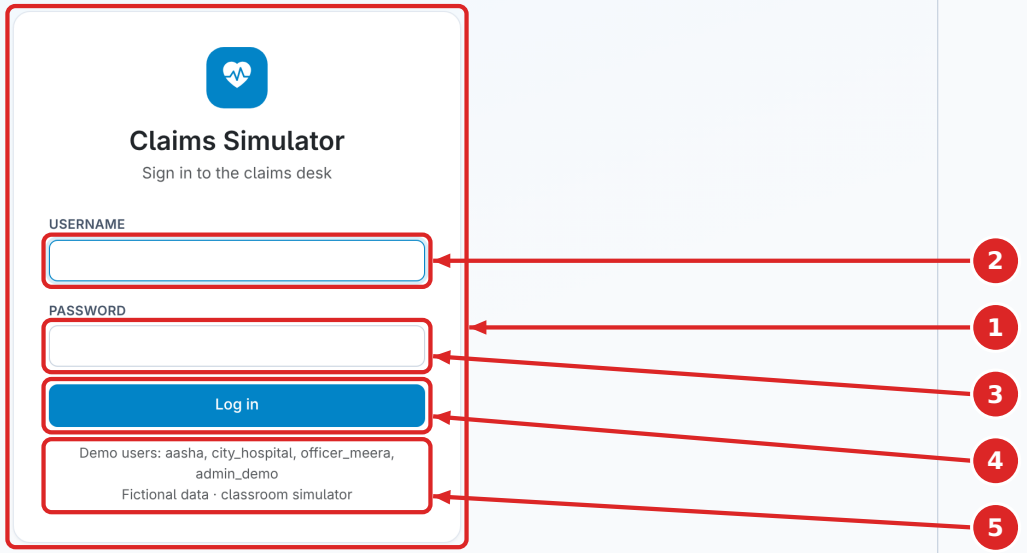
Every page also has: a search box (jumps to the claims list), your name and role, and Log out.

Try it: log in as aasha, officer_meera and admin_demo in three browsers and follow one claim.

Login

Anyone /login

The only public page; no self-registration



What this screen is for

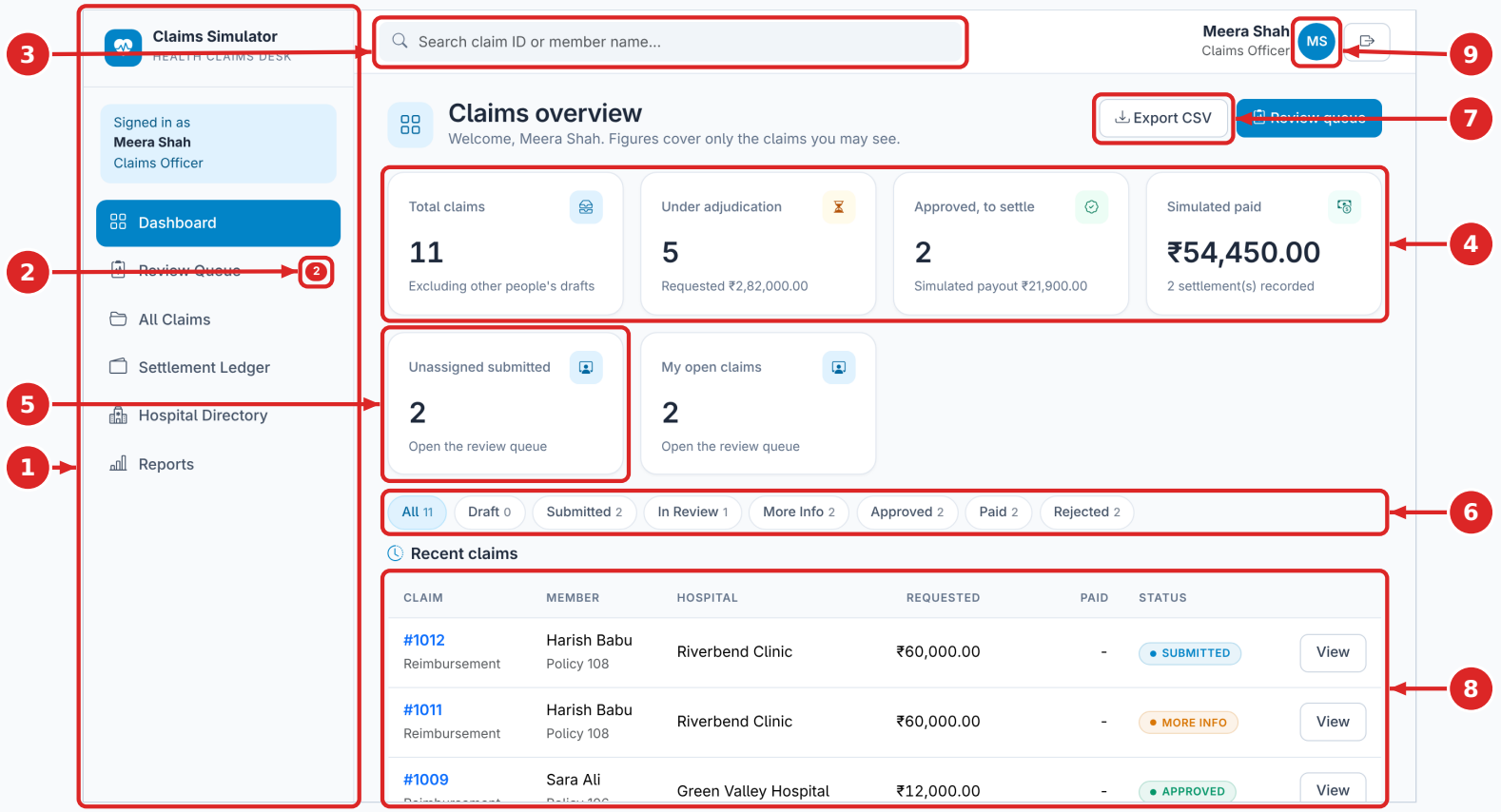
Entry point of the application. Nothing else can be opened without logging in: any other URL sends an anonymous visitor back here.

1. **Sign-in card**
   Only pre-created accounts can enter. Roles are assigned by the administrator, not by the user.
2. **Username**
   For example aasha, city_hospital, officer_meera or admin_demo.
3. **Password**
   Checked against a salted hash. A wrong user or password shows one generic error message.
4. **Log in**
   Starts a fresh session and opens the dashboard that matches the role.
5. **Demo hint**
   Lists one sample account per role.

Dashboard

Home page after login (shown here as a claims officer)

All roles /



What this screen is for

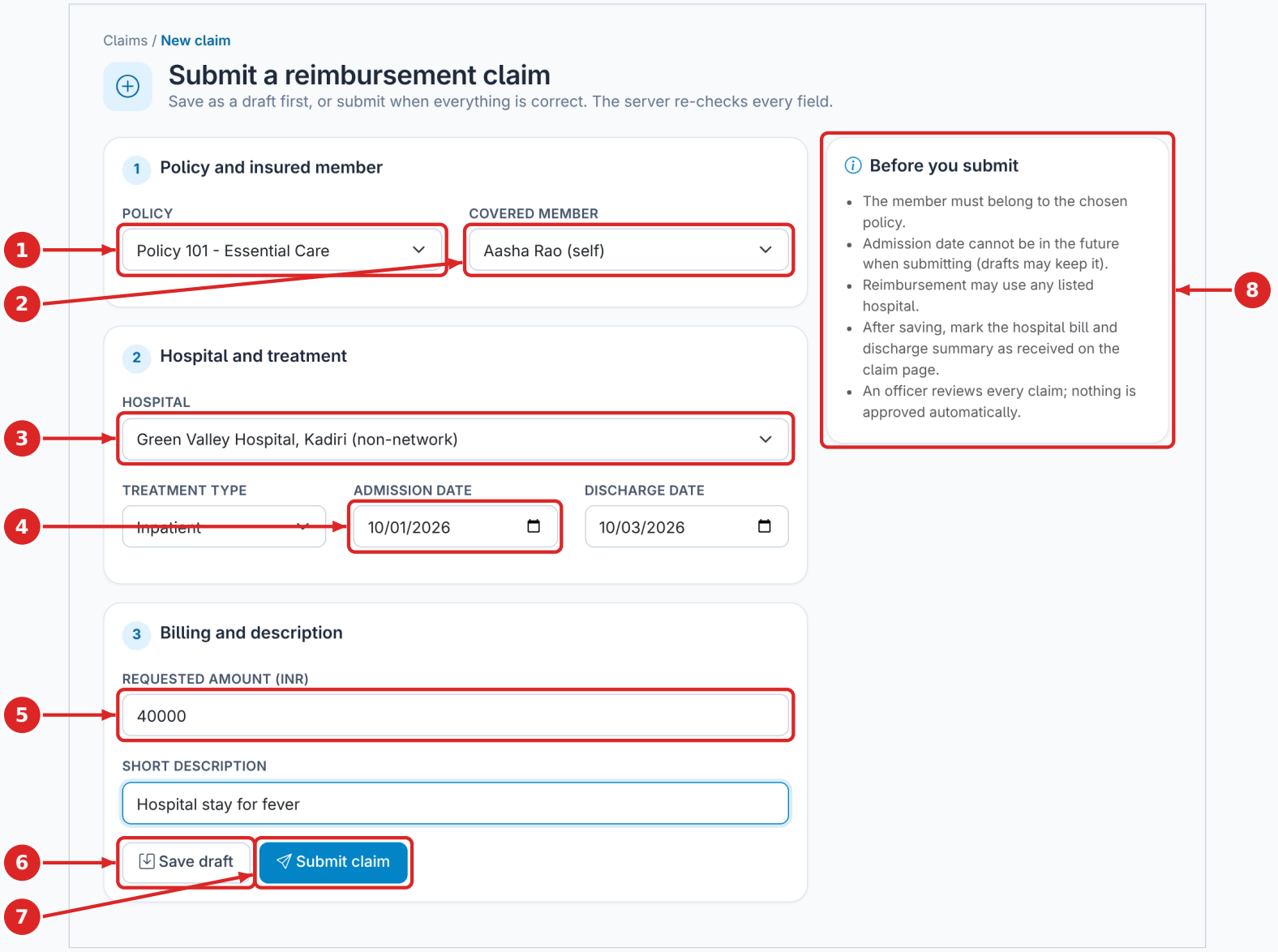
A quick picture of the work for the logged-in role. Every number counts only claims that role is allowed to see, so no data leaks between roles.

1. **Sidebar menu**
   Links depend on the role: officers see Review Queue, holders see My Policies, the admin sees Administration.
2. **Queue badge**
   Officer only: how many submitted claims are still unassigned.
3. **Search box**
   Type a claim ID or a member name; the claims list opens with that filter.
4. **Summary cards**
   Total claims, claims under adjudication (with requested value), approved claims waiting to be settled (with payout) and the simulated paid total. All come from the database.
5. **Officer cards**
   Only for officers: unassigned claims and your own open claims.
6. **Status tabs**
   One click lists all claims in that status; the small number is the count.
7. **Export CSV**
   Downloads the claims you may see (with current filters) as a spreadsheet.
8. **Recent claims**
   The 8 newest claims with amounts and a colour status pill. View opens the claim page.
9. **Your account**
   Who is logged in; the button beside it logs out.

New reimbursement claim

Created by the policyholder; the server re-checks every field

Policyholder /claims/new



What this screen is for

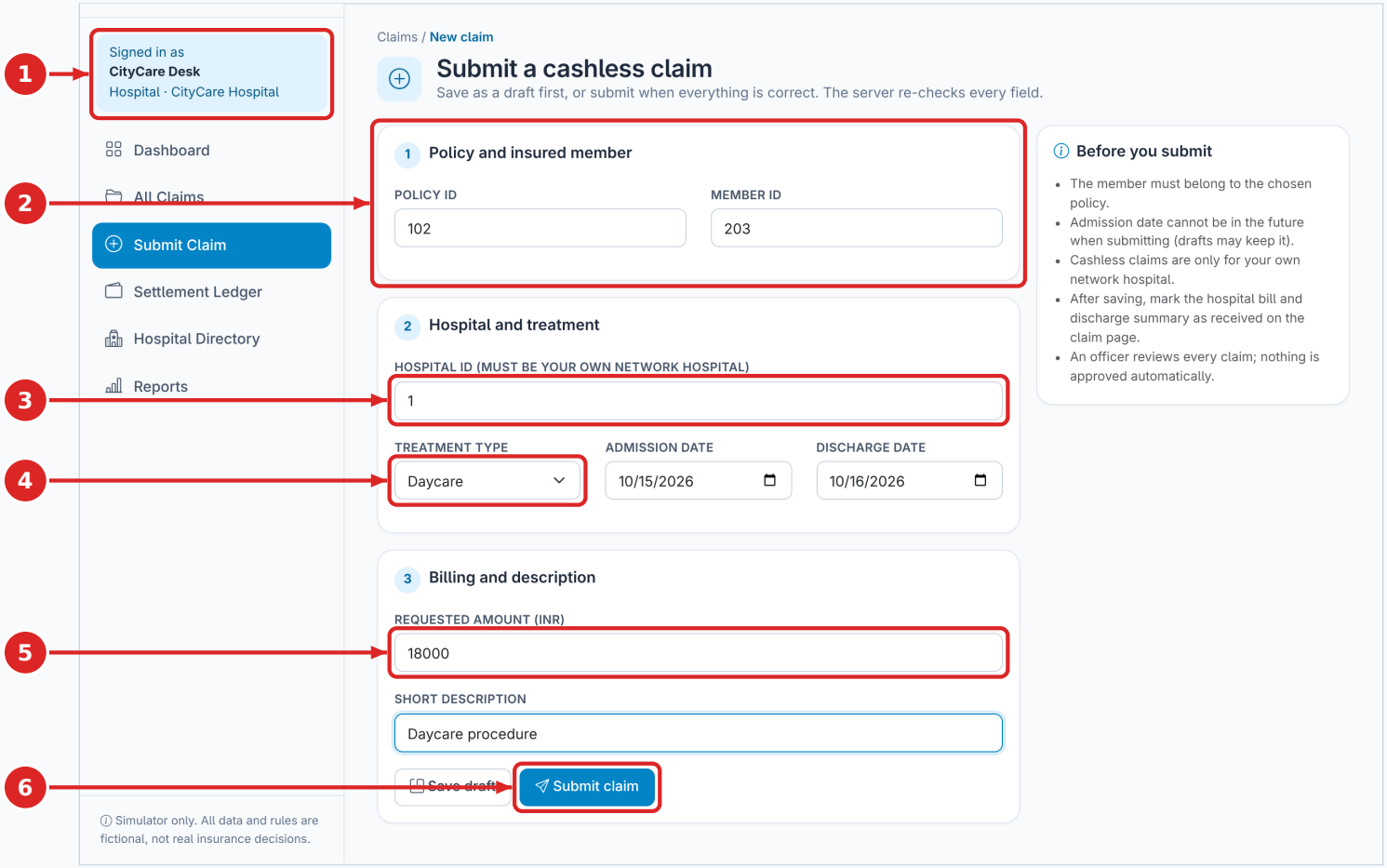
A reimbursement claim is requested by the policyholder after treatment. The claim type is decided by the role, never by a field in the form.

1. **Policy**
   Only the holder's own policies are listed.
2. **Covered member**
   The list follows the chosen policy. A member of another policy is refused by the server even if the form is forged.
3. **Hospital**
   Reimbursement may use any hospital, network or not.
4. **Dates**
   Admission cannot be in the future when submitting; discharge cannot be before admission.
5. **Requested amount**
   Must be above 0; stored as an exact decimal.
6. **Save draft**
   Keeps the claim as draft, visible only to its creator.
7. **Submit claim**
   Validates everything and moves draft -> submitted. An officer can now pick it up.
8. **Rules box**
   Short reminder of the checks the server applies.

New cashless claim

Hospital desk /claims/new

Created by a hospital desk for its own network hospital



What this screen is for

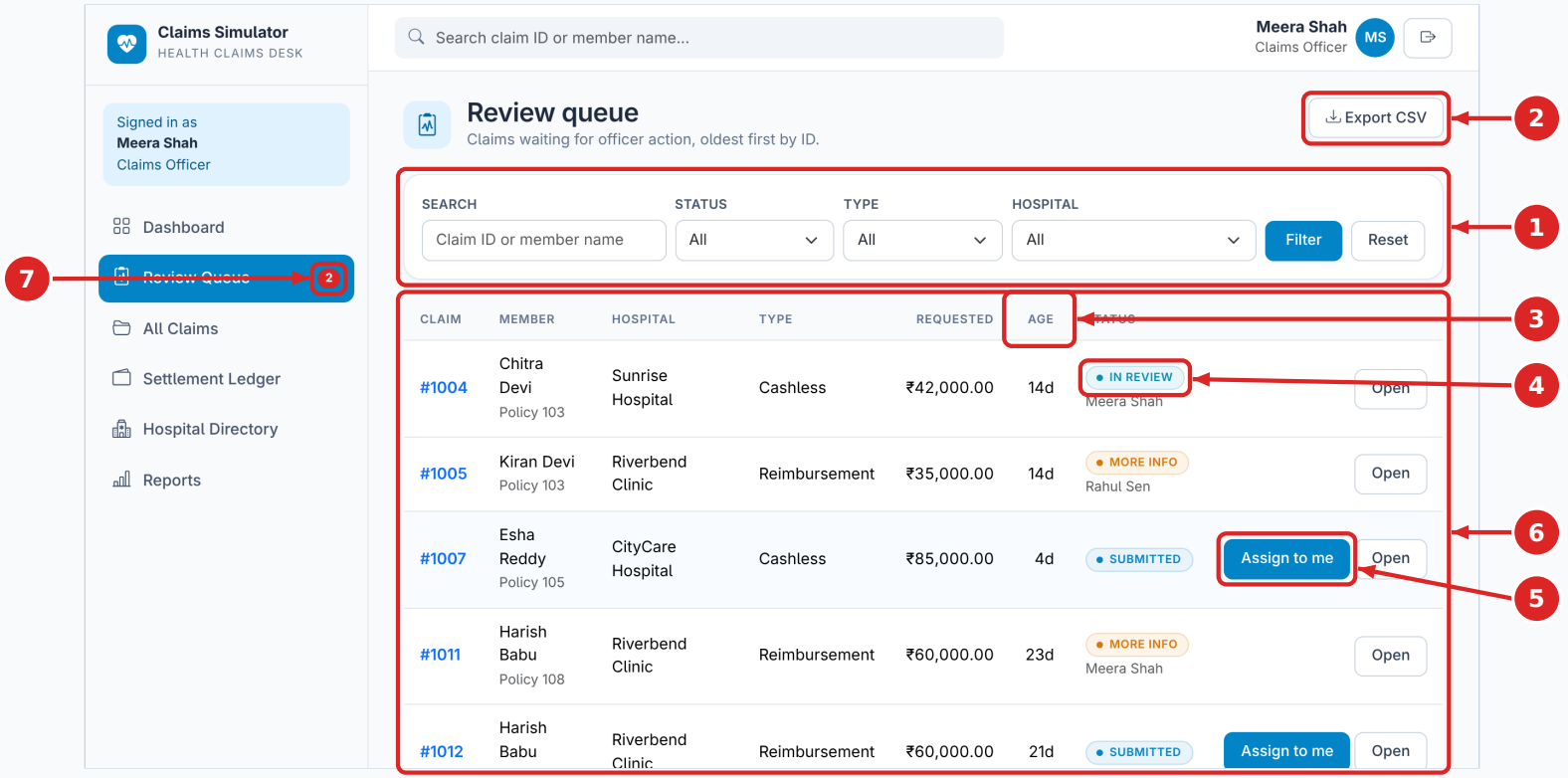
A cashless claim is raised by the hospital desk, not by the patient. This page proves the hospital-scope rule: the hospital cannot be changed to another one.

1. **Desk account**
   A hospital desk user is linked to exactly one hospital (users.hospital_id).
2. **Policy and member IDs**
   Typed in by the desk. The server checks that the member belongs to that policy.
3. **Hospital ID**
   Pre-filled with your own hospital. Any other ID is refused, and so is a hospital outside the network.
4. **Treatment type**
   Inpatient or daycare; the officer later checks that the plan covers it.
5. **Requested amount**
   Amount the hospital asks the insurer to pay directly.
6. **Submit claim**
   Creates the claim as submitted; the payee of a settlement will be the hospital.

Claim queue and search

Combined search and filters; officers can assign claims here

Claims officer /review



What this screen is for

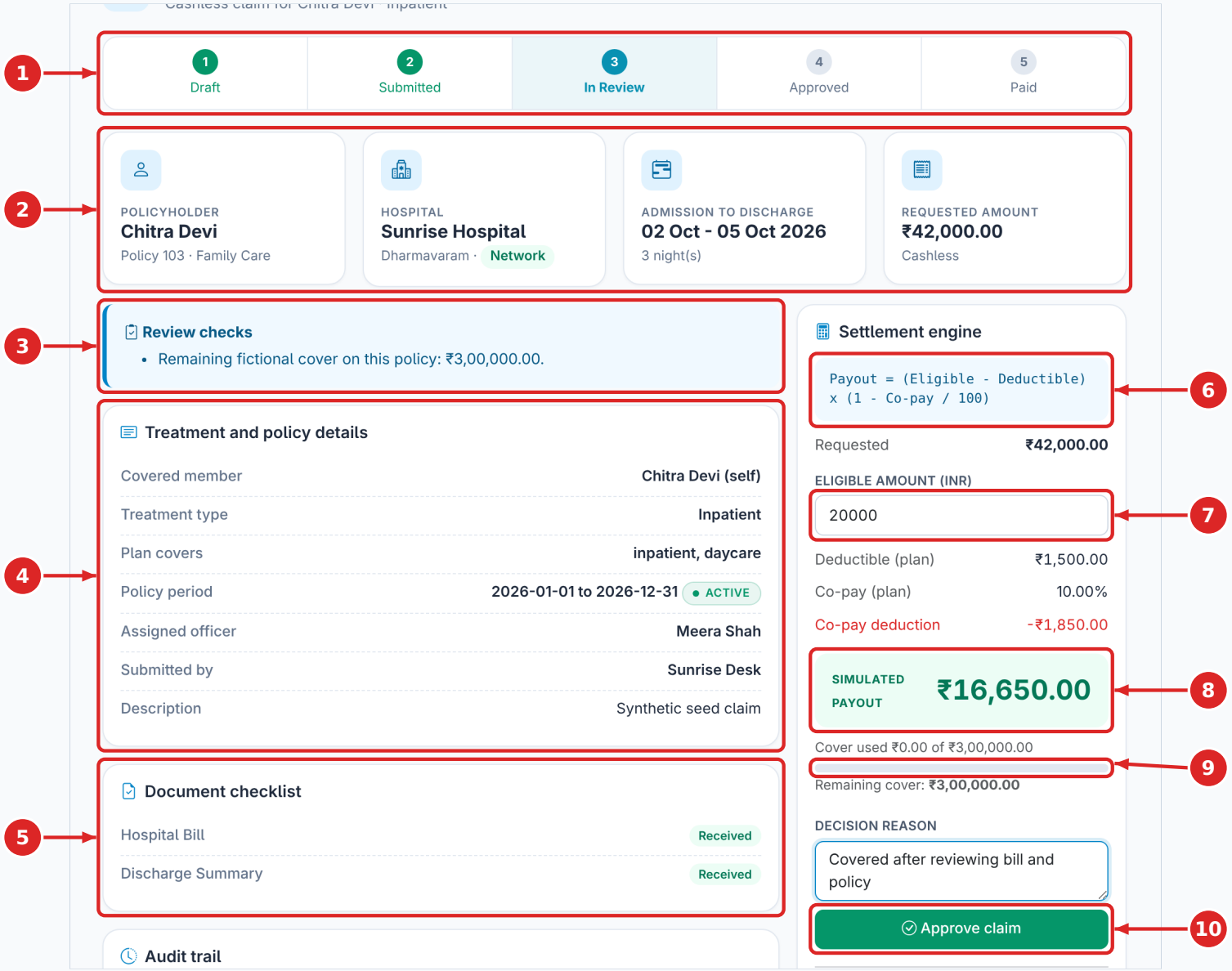
The officer's work queue shows claims that are submitted, in review or waiting for information. Other roles use the same list under 'All Claims', limited to their own records.

1. **Search and filters**
   Claim ID or member name, status, type and hospital work together. Reset clears them all.
2. **Export CSV**
   Downloads exactly the rows that match the filters.
3. **Age**
   Days since the claim was created. No legal deadlines are invented.
4. **Status pill**
   Colour shows the state; the assigned officer's name appears under it.
5. **Assign to me**
   Takes the claim. If two officers click at the same moment only the first succeeds.
6. **Claim rows**
   Click an ID or Open to see the claim page.
7. **Menu badge**
   Number of unassigned submitted claims.

Claim page (1/2): review the claim

Everything about one claim; the buttons depend on role and status

Claims officer /claims/1004



What this screen is for

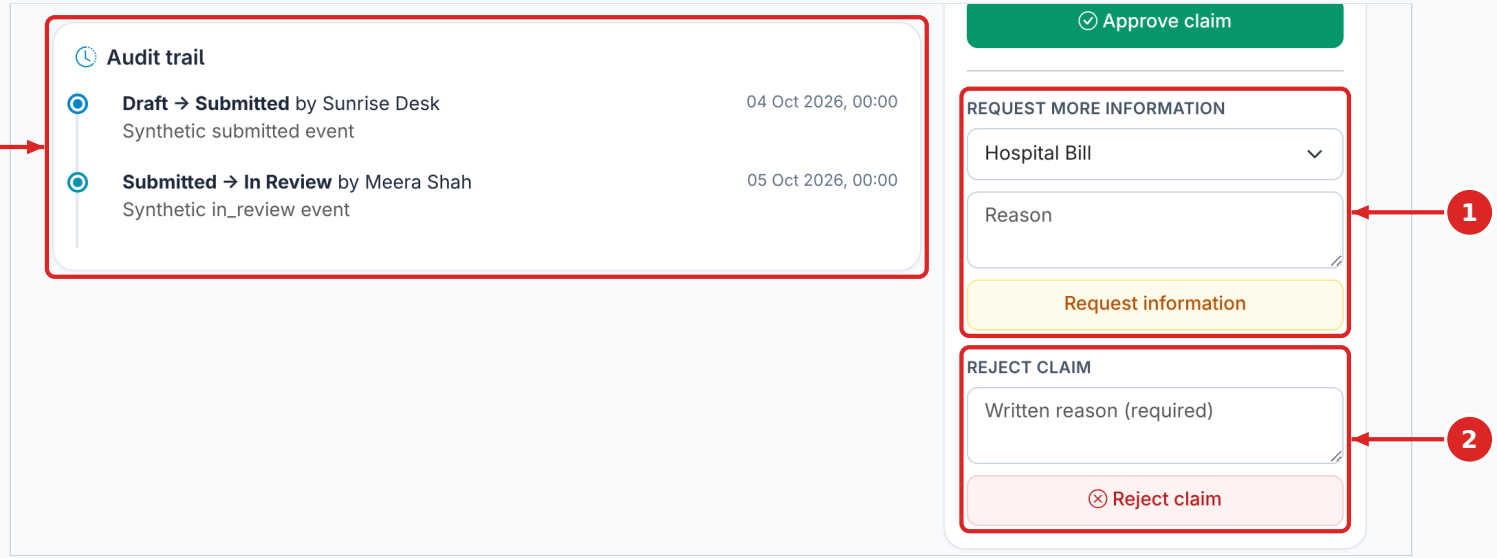
The most important screen. The officer reads the warnings, decides an eligible amount and approves. Nobody can approve automatically.

1. **Progress bar**
   Draft, Submitted, In Review, Approved, Paid. Rejected and More Info show in their own colour.
2. **Summary cards**
   Policyholder and plan, hospital (network or not), stay dates, requested amount.
3. **Review checks**
   Warnings for the officer: policy not active, treatment not covered, duplicate, missing document, cover left.
4. **Treatment and policy**
   Member, plan cover, policy period, assigned officer and who submitted.
5. **Document checklist**
   Hospital bill and discharge summary: received or missing (metadata only, no files).
6. **Settlement engine**
   Shows the formula used for the payout.
7. **Eligible amount**
   Typed by the officer: above 0, not above the request, not above remaining cover.
8. **Live payout**
   Updates while typing. The server recalculates with exact decimals.
9. **Cover used**
   Share of the annual limit already paid out on this policy.
10. **Approve claim**
    Needs a written reason. in_review -> approved, history row saved.

Claim page (2/2): other decisions and audit trail

Claims officer /claims/1004

Same page, lower part



What this screen is for

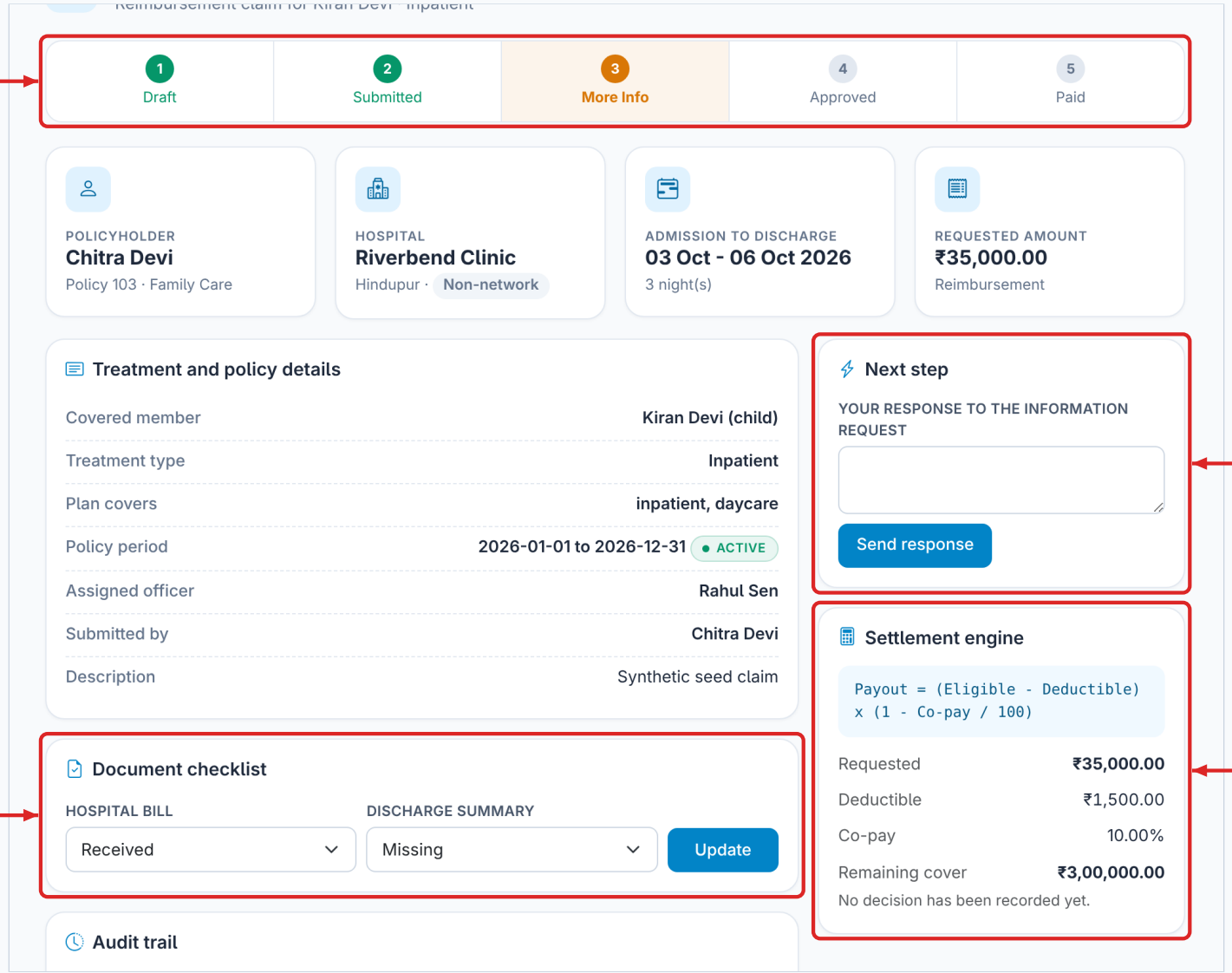
The officer has three outcomes: approve (previous slide), ask for more information, or reject. All of them need a reason, so the holder can always see why.

1. **Request information**
   Choose the missing document and give a reason: in_review -> more_info. The submitter is asked to respond.
2. **Reject claim**
   A written reason is mandatory. Rejected is final and cannot be reopened.
3. **Audit trail**
   Every status change: who, from -> to, when and the note. Stored in claim_history in the same transaction.

Claim page: submitter answers a request

Policyholder /claims/1005

The more-info cycle, seen by the policyholder



What this screen is for

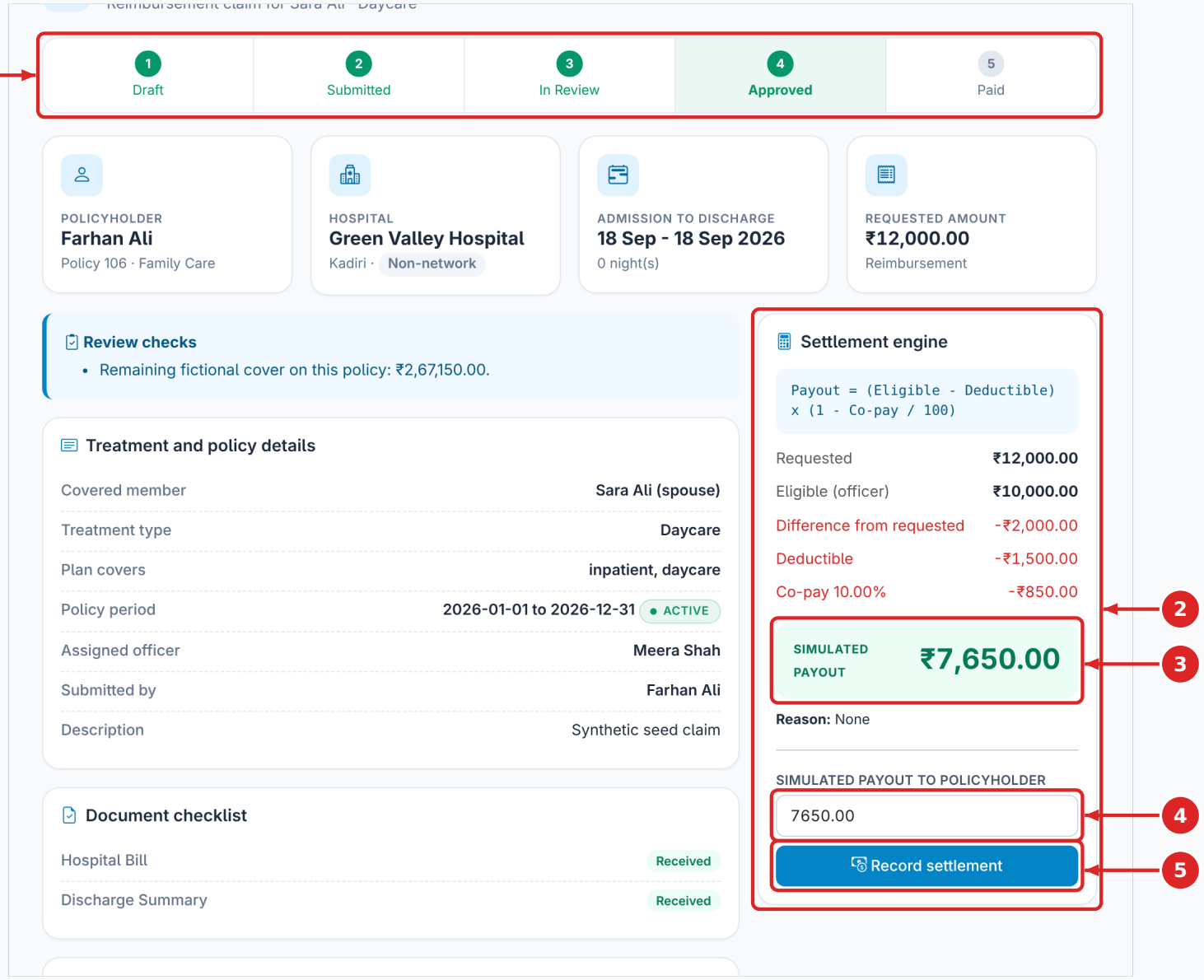
Only the original submitter can answer. Another holder, a hospital desk of a different hospital, or an officer cannot respond for them.

1. **Status: More Info**
   The claim is waiting for the submitter; the bar shows it in amber at the review step.
2. **Respond**
   The submitter writes a reply: more_info -> submitted. The assigned officer sees it again.
3. **Update checklist**
   The holder marks the missing document as received; the change is audited.
4. **Plan figures**
   The holder sees plan deductible, co-pay and remaining cover, but no officer tools.

Claim page: administrator records the settlement

Approved -> Paid, exactly once

Administrator /claims/1009



What this screen is for

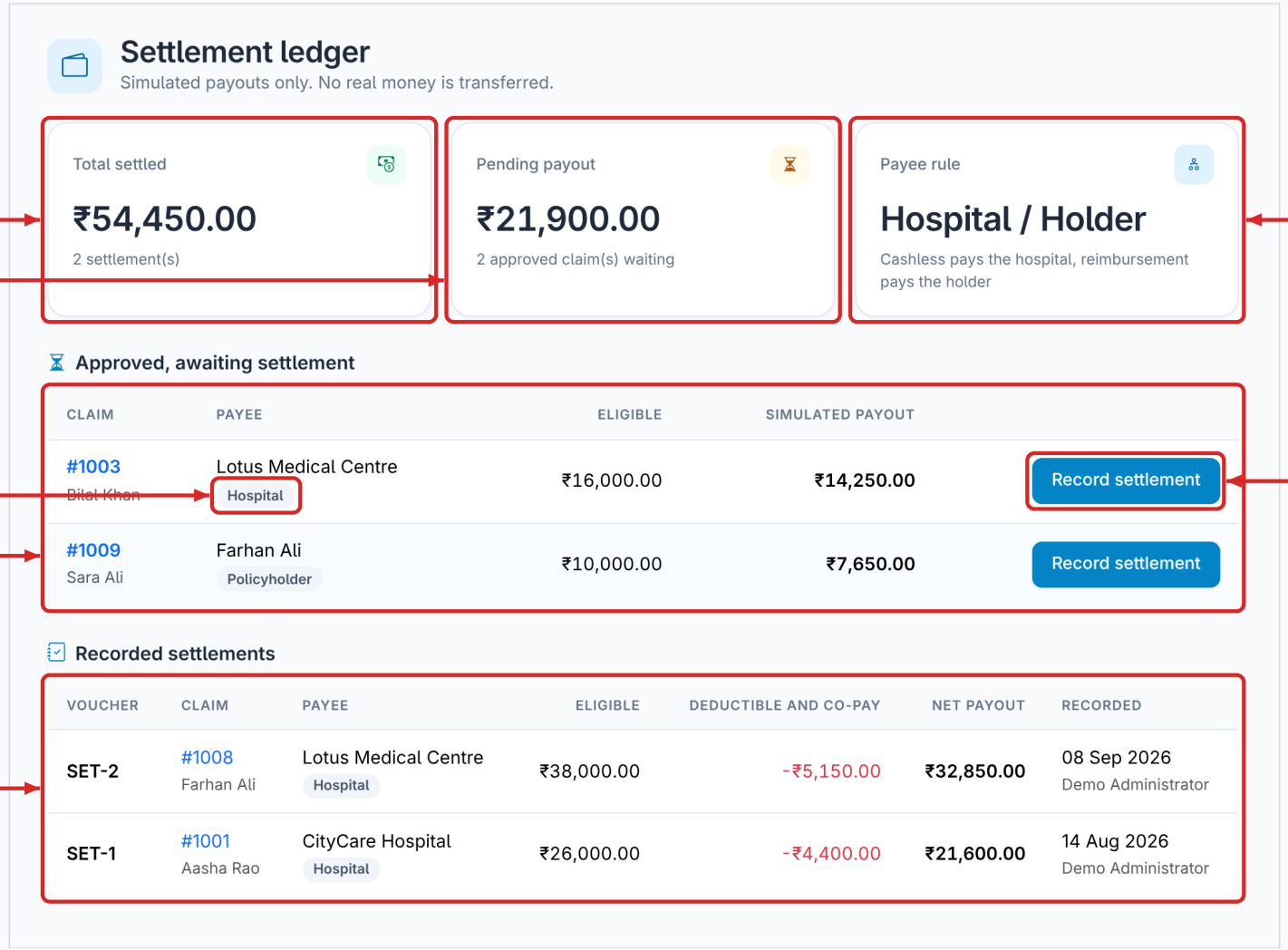
The administrator does not decide: the officer already did. The administrator only records the simulated payment.

1. **Approved stage**
   Only an approved claim can be settled.
2. **Final breakdown**
   Requested, eligible, deductible, co-pay and the simulated payout, plus the officer's reason.
3. **Net payout**
   Computed by the same formula as the officer's preview.
4. **Amount (read-only)**
   The server rejects any amount that differs from the computed payout.
5. **Record settlement**
   approved -> paid. One settlement row is created; payee is the policyholder (reimbursement) or the hospital (cashless). No real money moves.

Settlement ledger

All simulated payouts and the ones still waiting

All roles (admin settles) /settlements



What this screen is for

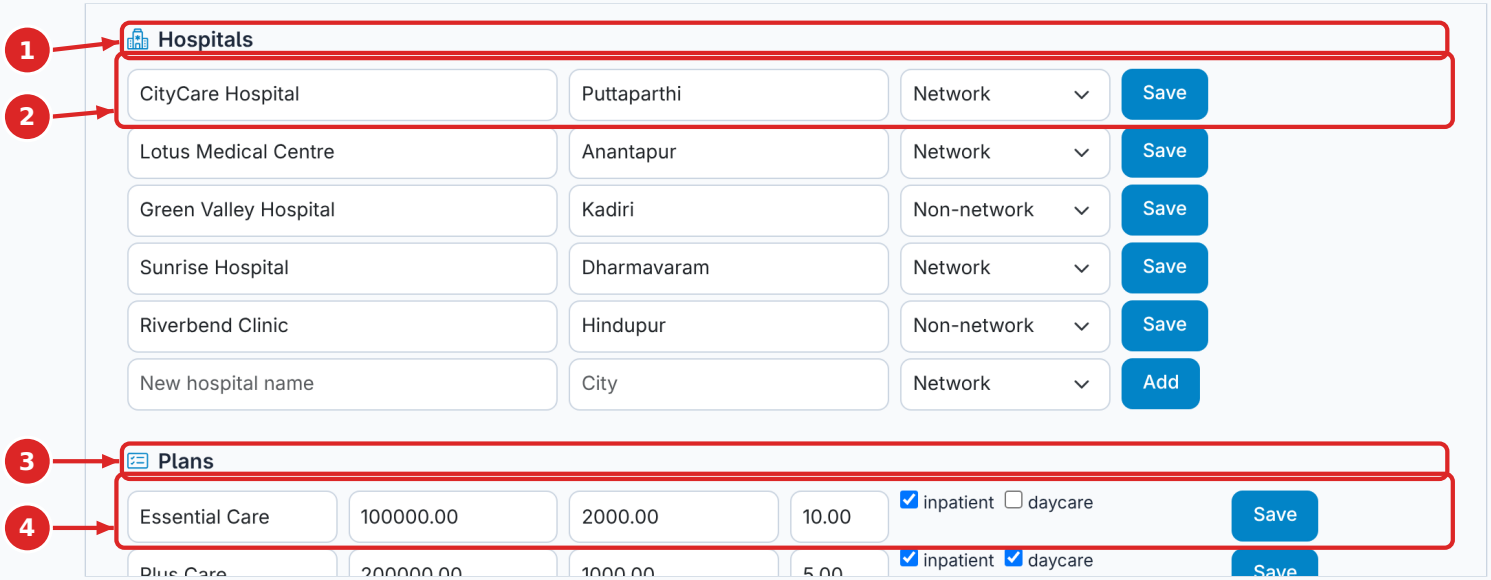
A bookkeeping view of the simulated money. Each role sees only the settlements of claims it is allowed to see.

1. **Total settled**
   Sum of all simulated payouts recorded.
2. **Pending payout**
   Approved claims that have no settlement yet, and what they will pay.
3. **Payee rule**
   Cashless pays the hospital; reimbursement pays the policyholder.
4. **Awaiting settlement**
   Each approved claim with its payee and computed payout.
5. **Record settlement**
   Administrator only: one click creates the settlement and marks the claim paid.
6. **Recorded settlements**
   Voucher SET-n, deductions, net payout, who recorded it and when.
7. **Payee type**
   Hospital or policyholder, decided by the claim type.

Administration (1/2): hospitals and plans

Reference data: add or edit, never delete

Administrator /admin



What this screen is for

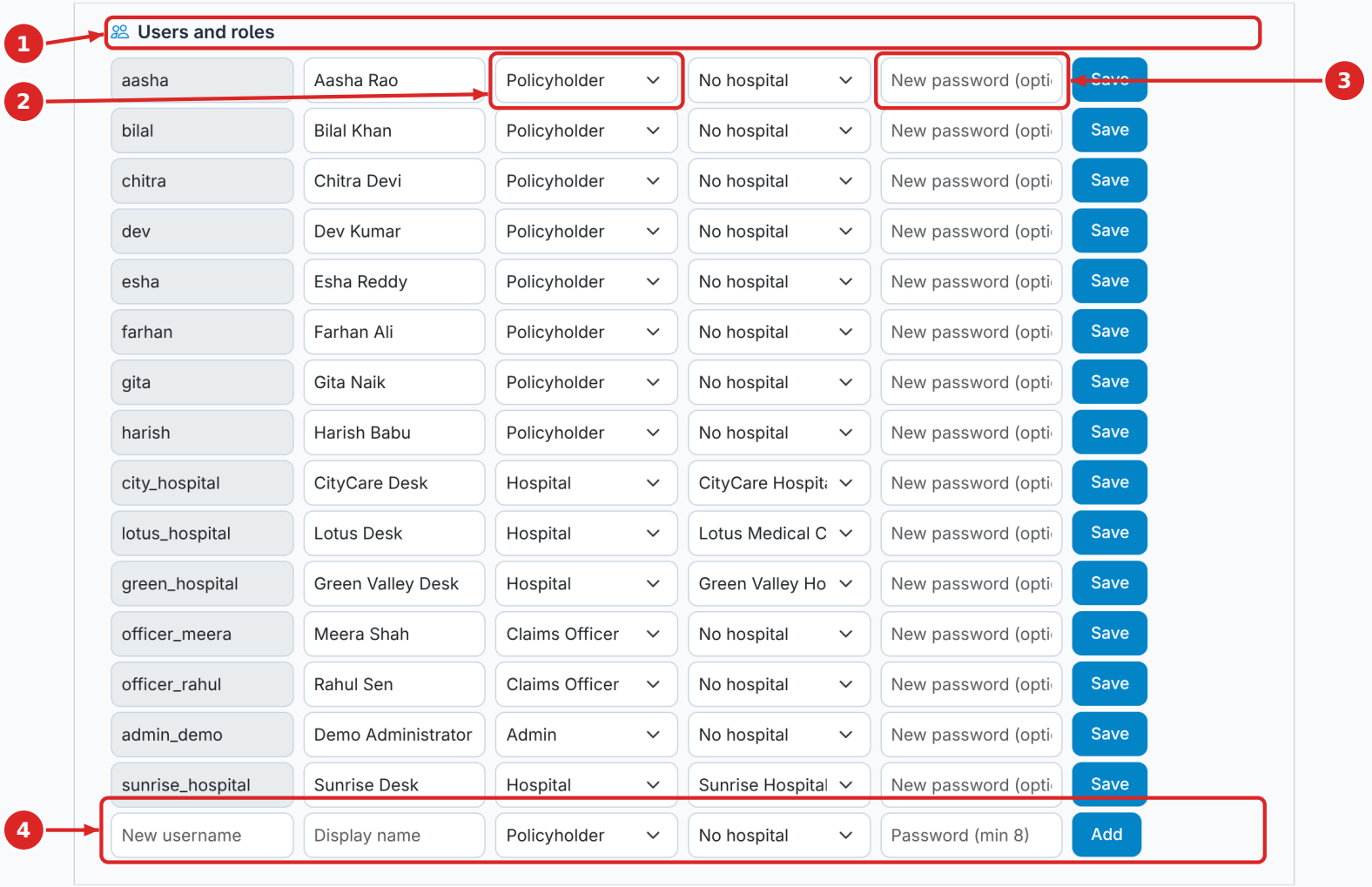
Only an administrator can open this page; every other role gets 403 (access denied), even with the URL typed in by hand.

1. **Hospitals**
   Name, city and network status. Network = cashless is allowed there.
2. **Edit row**
   Change a hospital and press Save. There is no delete, so old claims never lose their hospital.
3. **Plans**
   Annual limit, deductible, co-pay % and covered treatment types; used by the payout calculation.
4. **Plan row**
   Edit the numbers or the covered types and Save.

Administration (2/2): users and roles

Administrator /admin

Roles are assigned here, not by self-registration



What this screen is for

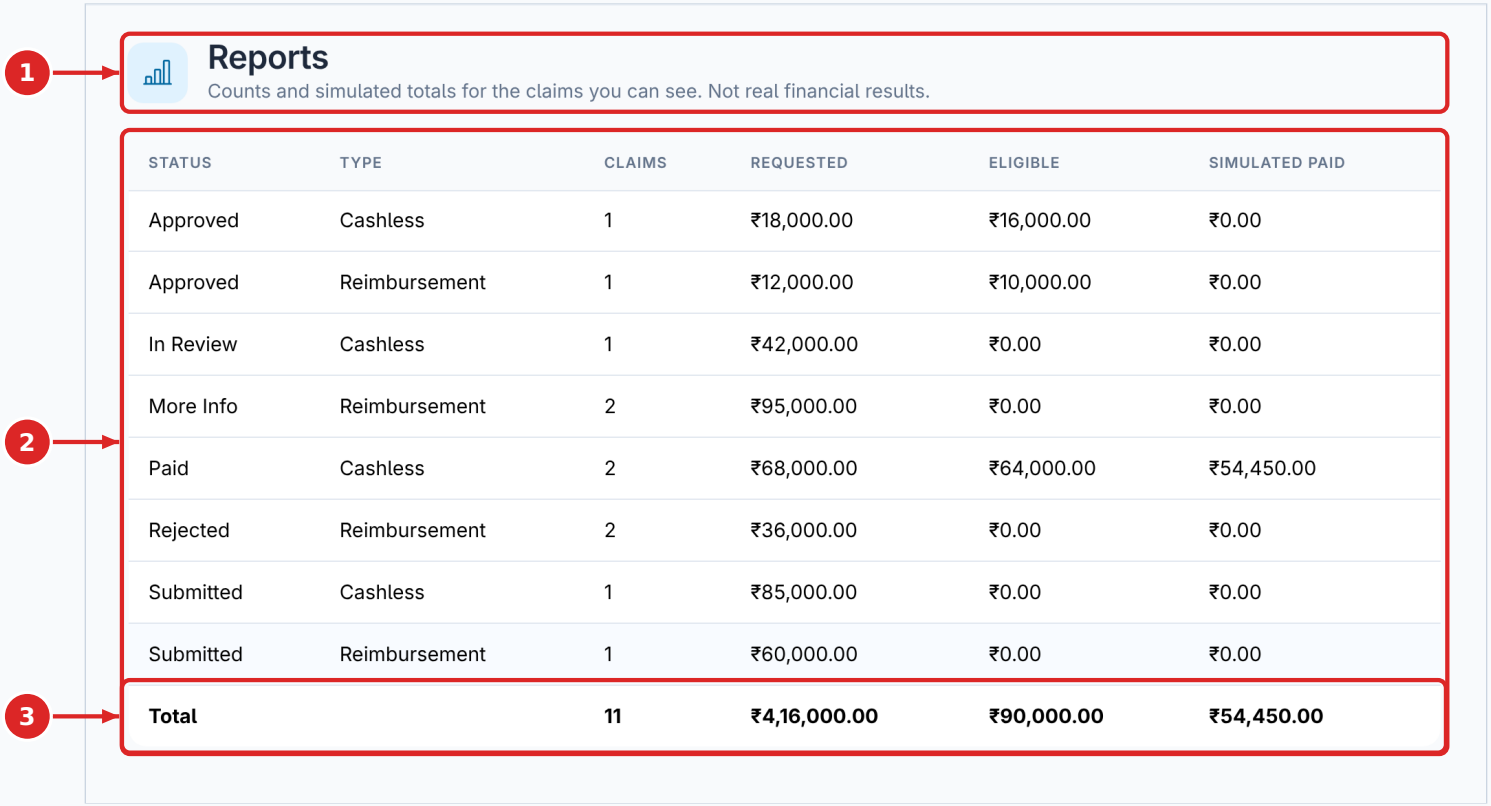
Role changes take effect on the next request, because every route checks the role again.

1. **Users and roles**
   All accounts. New users get a password of at least 8 characters, stored as a hash.
2. **Role**
   Change the role. A hospital desk must also get a hospital. You cannot change your own role.
3. **Password**
   Optional when editing: leave empty to keep the current password.
4. **Add row**
   The last row creates a new user.

Reports

Counts and simulated totals for the claims you may see

All roles | /reports



What this screen is for

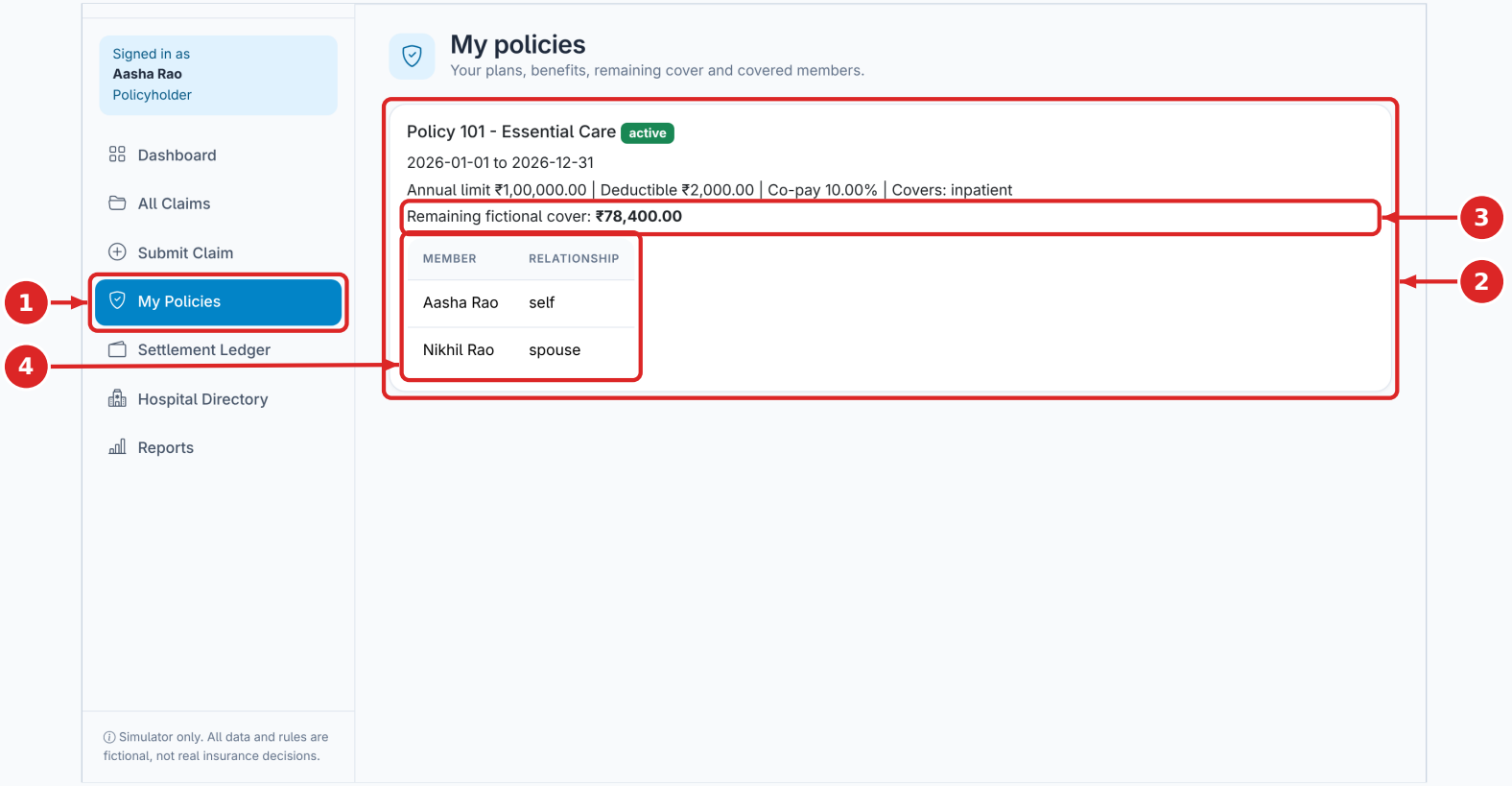
A holder sees only their own figures, a hospital desk only its own, officers and the admin see every non-draft claim.

1. **Scope note**
   The numbers are simulated, not real financial results.
2. **Status and type table**
   Claims grouped by status and type with requested, eligible and simulated paid totals.
3. **Totals**
   Sum of all rows; matches the filtered database records.

My policies

Holder only: plan benefits, remaining cover and members

Policyholder /policies



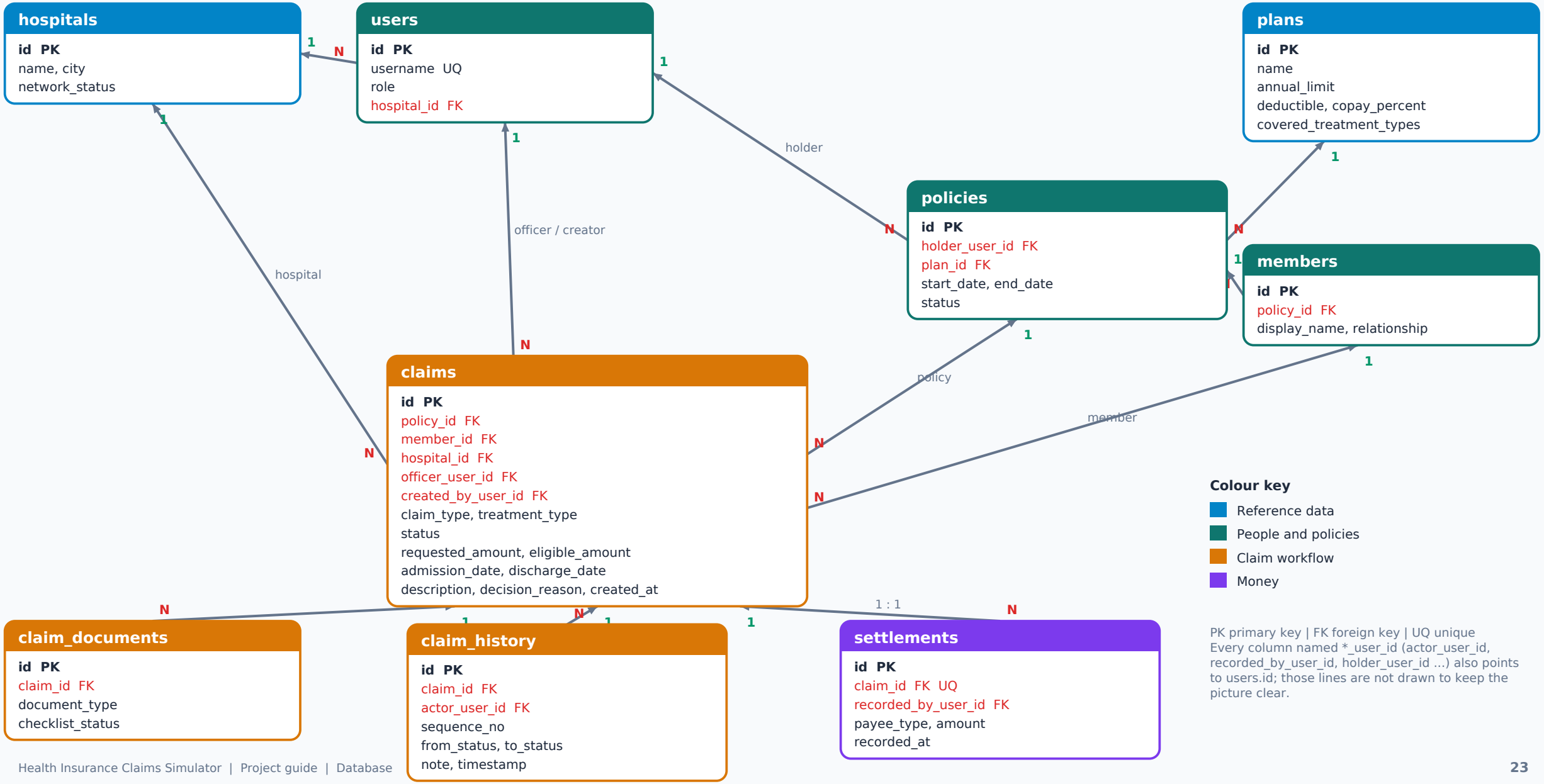
What this screen is for

Ownership comes from policies.holder_user_id, so a holder can never see another holder's policy.

1. **Menu item**
   Only policyholders have this menu entry.
2. **Policy card**
   Plan name, status, dates, limit, deductible, co-pay and covered treatment types.
3. **Remaining cover**
   Annual limit minus simulated payouts already paid on this policy.
4. **Covered members**
   The people a claim can be filed for under this policy.

8. Database schema: relationship diagram (ER)

9 tables. Each arrow points from the table holding the foreign key (N, many) to the table it refers to (1, one).



9. Table details (1/3): reference data and people

Column | type | key / constraint | meaning. PK primary key, FK foreign key, UQ unique, NN not null, CK check, IX in an index

hospitals — Reference

id	INT	**PK**	Hospital number
name	VARCHAR(120)	**NN**	Hospital name
city	VARCHAR(80)	**NN**	City
network_status	BOOL	**NN**	1 = cashless allowed

plans — Reference

id	INT	**PK**	Plan number
name	VARCHAR(80)	**NN**	e.g. Essential Care
covered_treatment_types	VARCHAR(100)	**NN**	inpatient | daycare, separated by |
annual_limit	DECIMAL(12,2)	**NN**	Yearly cover limit
deductible	DECIMAL(12,2)	**NN**	Amount not paid
copay_percent	DECIMAL(5,2)	**NN**	Holder's share in %

users — People

id	INT	**PK**	User number
username	VARCHAR(50)	**UQ NN**	Login name, unique
password_hash	VARCHAR(255)	**NN**	Salted hash, never plain text
display_name	VARCHAR(100)	**NN**	Name shown on screen
role	VARCHAR(20)	**NN CK**	policyholder | hospital | claims_office..
hospital_id	INT	**FK NULL**	-> hospitals.id (hospital desk only)

policies — People

id	INT	**PK**	Policy number
holder_user_id	INT	**FK NN**	-> users.id (the holder)
plan_id	INT	**FK NN**	-> plans.id
start_date	DATE	**NN**	Policy start
end_date	DATE	**NN**	Policy end
status	VARCHAR(20)	**NN**	active | expired

10. Table details (2/3): members and claims

Column | type | key / constraint | meaning. PK primary key, FK foreign key, UQ unique, NN not null, CK check, IX in an index

members

People

id	INT	**PK**	Member number
policy_id	INT	**FK NN**	-> policies.id
display_name	VARCHAR(100)	**NN**	Covered person
relationship	VARCHAR(30)	**NN**	self, spouse, child ...

claims

Workflow

id	INT	**PK**	Claim number (#1001 ...)
policy_id	INT	**FK NN IX**	-> policies.id
member_id	INT	**FK NN**	-> members.id (must belong to the polic..
hospital_id	INT	**FK NN**	-> hospitals.id
claim_type	VARCHAR(20)	**NN**	cashless | reimbursement
treatment_type	VARCHAR(20)	**NN**	inpatient | daycare
admission_date	DATE	**NN**	Admission
discharge_date	DATE	**NN**	Discharge
requested_amount	DECIMAL(12,2)	**NN CK>0**	Amount asked
eligible_amount	DECIMAL(12,2)	**NULL**	Officer's eligible amount
description	VARCHAR(255)	**NN**	Short description
decision_reason	VARCHAR(500)	**NULL**	Officer's written reason
status	VARCHAR(20)	**NN CK IX**	draft ... paid (7 states)
officer_user_id	INT	**FK NULL IX**	-> users.id (assigned officer)
created_by_user_id	INT	**FK NN**	-> users.id (submitter)
created_at	DATETIME	**NN**	Created / claim age

claims is the centre of the schema: every other workflow table points to it. requested_amount and eligible_amount use DECIMAL so money is exact.

10. Table details (3/3): documents, history and settlements

Column | type | key / constraint | meaning. PK primary key, FK foreign key, UQ unique, NN not null, CK check, IX in an index

claim_documents — Workflow

id	INT	**PK**	Row number
claim_id	INT	**FK NN UQ**	-> claims.id
document_type	VARCHAR(30)	**NN UQ**	hospital_bill | discharge_summary
checklist_status	VARCHAR(20)	**NN**	received | missing

settlements — Money

id	INT	**PK**	Voucher number (SET-n)
claim_id	INT	**FK NN UQ**	-> claims.id (one per claim)
payee_type	VARCHAR(20)	**NN**	hospital | policyholder
amount	DECIMAL(12,2)	**NN**	Computed payout
recorded_by_user_id	INT	**FK NN**	-> users.id (admin)
recorded_at	DATETIME	**NN**	When recorded

claim_history — Workflow

id	INT	**PK**	Row number
claim_id	INT	**FK NN UQ IX**	-> claims.id
sequence_no	INT	**NN UQ**	1, 2, 3 ... per claim
from_status	VARCHAR(20)	**NULL**	Previous status
to_status	VARCHAR(20)	**NN**	New status
actor_user_id	INT	**FK NN**	-> users.id (who did it)
note	VARCHAR(500)	**NN**	Reason / note
timestamp	DATETIME	**NN IX**	When

claim_history is append-only: rows are added for each change and never edited. settlements.claim_id is UNIQUE, so a claim can be paid only once.

11. Keys, constraints, indexes and how the database protects the rules

Database rules and server checks work together

Foreign keys (14)

claims -> policies, members, hospitals, users (officer, creator)
policies -> users (holder), plans
members -> policies
users -> hospitals
claim_documents, claim_history, settlements -> claims
claim_history.actor_user_id, settlements.recorded_by_user_id -> users

Unique rules

users.username
settlements.claim_id: only ONE settlement per claim
claim_documents (claim_id, document_type): one row per document
claim_history (claim_id, sequence_no): ordered, no gaps or duplicates

Check rules

users.role must be one of the 4 roles
claims.status must be one of the 7 states
claims.requested_amount > 0
(enforced by MySQL 8.0.16 or newer)

Indexes (speed)

ix_claims_policy_status (policy_id, status): claims of a policy, remaining cover
ix_claims_officer_status (officer_user_id, status): officer work queue
ix_history_claim_time (claim_id, timestamp): claim timeline

Rules the database cannot check alone, so the server checks them (services.py)

Member belongs to the chosen policy | hospital desk matches the claim's hospital | allowed status transitions and who may do them | two officers cannot take the same claim (one conditional UPDATE) | payout amount equals the computed payout | every change and its history row are saved in one transaction, or rolled back.

Sample data loaded by seed.py: 5 hospitals, 3 plans, 15 users, 8 policies, 11 members, 13 claims, 18 document rows, 33 history rows, 2 settlements. Import order: hospitals, plans, users, policies, members, claims, claim_documents, claim_history, settlements.

12. Run it and try it

Everything you need to demonstrate the project

1. **Install**

   ```
   pip install -r requirements.txt
   ```

2. **Configure**

   ```
   config.py: DB_USER, DB_PASSWORD (MySQL), database claims_db
   ```

3. **Create accounts (once)**

   ```
   flask --app wsgi seed   (type a demo password)
   ```

4. **Start**

   ```
   python run.py   then open http://127.0.0.1:5000
   ```

Demo path (matches the BRD final demonstration)

1. aasha: look at My policies, create a reimbursement claim, mark documents, submit
2. city_hospital: create a cashless claim; try another hospital ID and see it refused
3. officer_meera: assign claim, start review, ask for info, approve with eligible amount
4. chitra (holder of claim 1005): answer the information request
5. admin_demo: Settlement ledger, record the settlement, check the payee
6. aasha: open another holder's claim by URL (403); try search + filters
7. Restart the app: all data is still there (MySQL)

Sample logins (password = the one you typed at seeding)

Administrator	`admin_demo`
Claims officers	`officer_meera, officer_rahul`
Policyholders	`aasha, bilal, chitra, dev, esha, farhan, gita, harish`
Hospital desks	`city_hospital, lotus_hospital, green_hospital, sunrise_hospital`

Tests: python -m pytest (16 tests cover forged access, invalid transitions, member/policy mismatch, hospital scope, duplicate warning, payout, restart persistence, two-officer conflict, CSV export and the ledger).